



Lecture 23: Implementing GPT from scratch

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

- Implement a **complete GPT model** from scratch in PyTorch
- Build each component: **BPE tokenizer, embeddings, masked attention, transformer block**
- Write a **training loop** with proper optimization (AdamW, gradient clipping)
- Implement **text generation** with greedy decoding and sampling strategies
- Compare **temperature, top-k**, and **nucleus (top-p)** sampling

Announcements

Week 8: no classes

There are **no classes February 23--27** (instructor away). Use this time to work on your **final project** and the optional Assignment 5 (Build GPT).

After the break

Week 9 covers agents and tool use, mixture of experts, and ethics/safety. The final project is due **March 9**.

What we are building today

A complete mini-GPT

We will implement every component of a GPT language model from scratch -- small enough to train on a laptop, but architecturally identical to GPT-2:

1. **Tokenization** with Byte-Pair Encoding (tiktoken)
2. **Token + position embeddings**
3. **Masked multi-head attention**
4. **Transformer decoder blocks** (pre-norm with residual connections)
5. **Language model head** for next-token prediction
6. **Training loop** with AdamW and gradient clipping
7. **Text generation** with multiple sampling strategies

Setup and hyperparameters

Required libraries and configuration

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 from torch.utils.data import Dataset, DataLoader
5 import tiktoken # OpenAI's BPE tokenizer
6
7 device = 'cuda' if torch.cuda.is_available() else 'cpu'
8
9 config = {
10     'vocab_size': 50257,      # GPT-2 vocabulary size
11     'd_model': 384,          # Embedding dimension
12     'n_layers': 6,           # Number of transformer blocks
13     'n_heads': 6,            # Number of attention heads
14     'max_seq_len': 256,      # Maximum sequence length
15     'dropout': 0.1,
```

continued...

Setup and hyperparameters

Required libraries and configuration

```
16     'batch_size': 32,  
17     'learning_rate': 3e-4,  
18     'num_epochs': 10  
19 }
```

...continued

Tokenization with BPE

Byte-pair encoding recap

BPE starts with individual characters and iteratively merges the most frequent adjacent pairs into new tokens. This produces a subword vocabulary that handles common words as single tokens and rare words as sequences of subword pieces.

Using tiktoken (GPT-2's tokenizer)

```
1 tokenizer = tiktoken.get_encoding("gpt2")
2
3 text = "Hello, how are you doing today?"
4 tokens = tokenizer.encode(text)
5 # [15496, 11, 703, 389, 345, 1804, 1909, 30]
6
7 # Decode back to text
8 decoded = tokenizer.decode(tokens) # "Hello, how are you doing
9 today?"
10
11 # Inspect individual tokens
```

Tokenization with BPE

Using tiktoken (GPT-2's tokenizer)

```
12 | print(f" {tid}: '{tokenizer.decode([tid])}'")
```

...continued

Creating a text dataset

Sliding window over tokenized text

```
1 class TextDataset(Dataset):
2     def __init__(self, text_file, tokenizer, max_seq_len):
3         with open(text_file, 'r', encoding='utf-8') as f:
4             text = f.read()
5             self.tokens = tokenizer.encode(text)
6             self.max_seq_len = max_seq_len
7
8     def __len__(self):
9         return len(self.tokens) - self.max_seq_len
10
11    def __getitem__(self, idx):
12        chunk = self.tokens[idx : idx + self.max_seq_len + 1]
13        x = torch.tensor(chunk[:-1], dtype=torch.long) # Input
```

continued...

Creating a text dataset

Sliding window over tokenized text

```
14         y = torch.tensor(chunk[1:], dtype=torch.long) # Target
15         return x, y
16
17     dataset = TextDataset('shakespeare.txt', tokenizer,
18                           config['max_seq_len'])
19     dataloader = DataLoader(dataset, batch_size=config['batch_size'],
20                             shuffle=True)
```

...continued

Why offset by 1?

The target at each position is the *next* token. For input `[A, B, C, D]`, the targets are `[B, C, D, E]`. This is the autoregressive training signal.

Token and position embeddings

Embedding layer implementation

```
1 class Embeddings(nn.Module):
2     def __init__(self, vocab_size, d_model, max_seq_len, dropout):
3         super().__init__()
4         self.token_embed = nn.Embedding(vocab_size, d_model)
5         self.pos_embed = nn.Embedding(max_seq_len, d_model)
6         self.dropout = nn.Dropout(dropout)
7
8     def forward(self, x):
9         seq_len = x.size(1)
10        tok_emb = self.token_embed(x)                # (B,
11        pos_emb = self.pos_embed(torch.arange(seq_len,
12        device=x.device)) # (T, D)
13        return self.dropout(tok_emb + pos_emb)      #
14        Broadcasting adds positions
```

Masked multi-head attention

Attention with causal masking (part 1: projections)

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_model, n_heads, dropout):
3         super().__init__()
4         assert d_model % n_heads == 0
5         self.n_heads = n_heads
6         self.head_dim = d_model // n_heads
7
8         self.q_linear = nn.Linear(d_model, d_model)
9         self.k_linear = nn.Linear(d_model, d_model)
10        self.v_linear = nn.Linear(d_model, d_model)
11        self.out_linear = nn.Linear(d_model, d_model)
12        self.dropout = nn.Dropout(dropout)
```

continued...

Masked multi-head attention

Attention with causal masking (part 1: projections)

```
13
14     def forward(self, x, mask=None):
15         B, T, D = x.shape
16         Q = self.q_linear(x).view(B, T, self.n_heads,
17 self.head_dim).transpose(1, 2)
18         K = self.k_linear(x).view(B, T, self.n_heads,
19 self.head_dim).transpose(1, 2)
20         V = self.v_linear(x).view(B, T, self.n_heads,
21 self.head_dim).transpose(1, 2)
22         # Q, K, V shape: (B, n_heads, T, head_dim)
```

...continued

Attention computation

Attention with causal masking (part 2: scores and output)

```
1         # Scaled dot-product attention
2         scores = torch.matmul(Q, K.transpose(-2, -1)) /
   (self.head_dim ** 0.5)
3
4         if mask is not None:
5             scores = scores.masked_fill(mask == 0, float('-inf'))
6
7         attn_weights = F.softmax(scores, dim=-1)
8         attn_weights = self.dropout(attn_weights)
9
10        # Apply attention to values and concatenate heads
11        out = torch.matmul(attn_weights, V) # (B,
   n_heads, T, head_dim)
12        out = out.transpose(1, 2).contiguous().view(B, T, D) # (B,
   T, D)
13        return self.out_linear(out)
```


Feed-forward network and transformer block

The two sub-layers of each GPT block

```
1 class FeedForward(nn.Module):
2     def __init__(self, d_model, dropout):
3         super().__init__()
4         self.net = nn.Sequential(
5             nn.Linear(d_model, 4 * d_model),
6             nn.GELU(),                                # GPT uses GELU, not
ReLU
7             nn.Linear(4 * d_model, d_model),
8             nn.Dropout(dropout)
9         )
10    def forward(self, x):
11        return self.net(x)
```

continued...

Feed-forward network and transformer block

The two sub-layers of each GPT block

```
12
13 class TransformerBlock(nn.Module):
14     def __init__(self, d_model, n_heads, dropout):
15         super().__init__()
16         self.ln1 = nn.LayerNorm(d_model)
17         self.attention = MultiHeadAttention(d_model, n_heads,
18 dropout)
19         self.ln2 = nn.LayerNorm(d_model)
20         self.ffn = FeedForward(d_model, dropout)
21
22     def forward(self, x, mask):
23         x = x + self.attention(self.ln1(x), mask)    # Pre-norm +
24 residual
```

...continued...

Feed-forward network and transformer block

The two sub-layers of each GPT block

```
23         x = x + self.ffn(self.ln2(x))           # Pre-norm +  
    residual  
24     return x
```

...continued

Complete GPT model

Assembling all components

```
1 class GPT(nn.Module):
2     def __init__(self, vocab_size, d_model, n_layers, n_heads,
3       max_seq_len, dropout):
4         super().__init__()
5         self.max_seq_len = max_seq_len
6         self.embeddings = Embeddings(vocab_size, d_model,
7       max_seq_len, dropout)
8         self.blocks = nn.ModuleList([
9           TransformerBlock(d_model, n_heads, dropout) for _ in
10        range(n_layers)
11        ])
12        self.ln_f = nn.LayerNorm(d_model)
13        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
14        self.apply(self._init_weights)
```

continued...

Complete GPT model

Assembling all components

```
13     def _init_weights(self, module):
14         if isinstance(module, nn.Linear):
15             torch.nn.init.normal_(module.weight, mean=0.0,
std=0.02)
16             if module.bias is not None:
17                 torch.nn.init.zeros_(module.bias)
18         elif isinstance(module, nn.Embedding):
19             torch.nn.init.normal_(module.weight, mean=0.0,
std=0.02)
```

...continued

GPT forward pass

Forward pass with optional loss computation

```
1     def forward(self, x, targets=None):
2         seq_len = x.size(1)
3         mask = create_causal_mask(seq_len, x.device)
4
5         x = self.embeddings(x)
6         for block in self.blocks:
7             x = block(x, mask)
8         x = self.ln_f(x)
9         logits = self.lm_head(x)          # (batch, seq_len,
vocab_size)
10
11         loss = None
12         if targets is not None:
13             loss = F.cross_entropy(
14                 logits.view(-1, logits.size(-1)),
15                 targets.view(-1)
```

continued...

GPT forward pass

Forward pass with optional loss computation

```
17
```

```
    return logits, loss
```

...continued

Parameter count for our mini-GPT

With `d_model=384` , `n_layers=6` , `n_heads=6` , and `vocab_size=50257` , this model has approximately **30 million parameters** -- small enough to train on a single GPU in a few hours.

Training loop

Training with AdamW and gradient clipping

```
1  model = GPT(  
2      config['vocab_size'], config['d_model'], config['n_layers'],  
3      config['n_heads'], config['max_seq_len'], config['dropout']  
4  ).to(device)  
5  
6  optimizer = torch.optim.AdamW(  
7      model.parameters(), lr=config['learning_rate'],  
8      betas=(0.9, 0.95), weight_decay=0.1  
9  )  
10  
11  model.train()  
12  for epoch in range(config['num_epochs']):  
13      total_loss = 0  
14      for x, y in dataloader:
```

continued...

Training loop

Training with AdamW and gradient clipping

```
15         x, y = x.to(device), y.to(device)
16         logits, loss = model(x, targets=y)
17         optimizer.zero_grad()
18         loss.backward()
19         torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
20         optimizer.step()
21         total_loss += loss.item()
22     print(f"Epoch {epoch+1}, Loss: {total_loss /
len(dataloader):.4f}")
```

...continued

Training best practices

Five key techniques for stable GPT training

Technique	Why it matters
Gradient clipping (max norm 1.0)	Prevents exploding gradients that destabilize training
AdamW optimizer	Decoupled weight decay; better than standard Adam for transformers
Learning rate warmup	Gradually increase LR over first ~1,000 steps to avoid early instability
Cosine LR decay	Smoothly reduce LR after warmup for fine-grained convergence
Mixed precision (float16/bfloat16)	2--3x speedup on modern GPUs with minimal quality loss

Practical advice

If loss is not decreasing: check learning rate (try $1e-4$ to $3e-4$), verify data pipeline outputs correct input/target pairs, and watch for NaN values. If you run out of memory: reduce batch size or sequence length first, then try gradient accumulation.

Greedy decoding

Generating text by always picking the most likely token

```
1  @torch.no_grad()
2  def generate_greedy(model, tokenizer, prompt, max_new_tokens=50):
3      model.eval()
4      tokens = tokenizer.encode(prompt)
5      x = torch.tensor([tokens], dtype=torch.long, device=device)
6
7      for _ in range(max_new_tokens):
8          x_crop = x[:, -model.max_seq_len:]
9          logits, _ = model(x_crop)
10         logits = logits[:, -1, :] # Last position
11         only
12         next_token = torch.argmax(logits, dim=-1, keepdim=True)
13         x = torch.cat([x, next_token], dim=1)
14
15     return tokenizer.decode(x[0].tolist())
```

Sampling strategies

Three ways to add controlled randomness

- **Temperature (T):** Scale logits by $1/T$ before softmax. $T < 1$ sharpens the distribution (more conservative); $T > 1$ flattens it (more creative).
- **Top-k:** Sample only from the k most probable tokens. Typical: $k = 40$.
- **Nucleus (top-p):** Sample from the smallest set of tokens whose cumulative probability exceeds p . Typical: $p = 0.9$ or 0.95 .

Comparison

Strategy	Pros	Cons
Greedy	Deterministic, fast	Repetitive, boring
Temperature	Simple control knob	Can produce nonsense at high T
Top-k	Filters unlikely tokens	Fixed k may be too broad or narrow
Nucleus (top-p)	Adapts to distribution shape	Slightly more complex

Implementing sampling

Top-k and nucleus sampling in PyTorch

```
1 def sample_next_token(logits, temperature=1.0, top_k=None,
2   top_p=None):
3     logits = logits / temperature
4
5     if top_k is not None:
6         top_k = min(top_k, logits.size(-1))
7         threshold = torch.topk(logits, top_k)[0][..., -1, None]
8         logits[logits < threshold] = float('-inf')
9
10    if top_p is not None:
11        sorted_logits, sorted_idx = torch.sort(logits,
12        descending=True)
13        cumprobs = torch.cumsum(F.softmax(sorted_logits, dim=-1),
14        dim=-1)
15        remove = cumprobs > top_p
16        remove[..., 1:] = remove[..., :-1].clone()
17        remove[0] = False
```

continued...

Implementing sampling

Top-k and nucleus sampling in PyTorch

```
17     probs = F.softmax(logits, dim=-1)  
18     return torch.multinomial(probs, num_samples=1)
```

...continued

Complete generation function

Putting it all together

```
1  @torch.no_grad()
2  def generate(model, tokenizer, prompt, max_new_tokens=100,
3              temperature=0.8, top_k=40, top_p=0.9):
4      model.eval()
5      tokens = tokenizer.encode(prompt)
6      x = torch.tensor([tokens], dtype=torch.long, device=device)
7
8      for _ in range(max_new_tokens):
9          x_crop = x[:, -model.max_seq_len:]
10         logits, _ = model(x_crop)
11         next_token = sample_next_token(
12             logits[0, -1, :], temperature=temperature, top_k=top_k,
13             top_p=top_p
14         )
15         x = torch.cat([x, next_token.unsqueeze(0)], dim=1)
```

continued...

Complete generation function

Putting it all together

```
15  
16     return tokenizer.decode(x[0].tolist())  
17  
18     # Generate text  
19     print(generate(model, tokenizer, "Once upon a time"))
```

...continued

Debugging and common issues

Problems you will encounter

Problem	Likely cause	Fix
Loss not decreasing	LR too high/low, data bug	Try LR in $[1e-4, 3e-4]$; verify x/y offset
Out of memory	Batch/sequence too large	Reduce batch size; use gradient accumulation
Poor generation quality	Undertrained	Train longer; use more/better data
Repetitive output	Greedy decoding or low temperature	Use nucleus sampling ($p = 0.9$)
NaN loss	Numerical instability	Add gradient clipping; check for empty batches

Extensions for the ambitious

Ways to enhance your implementation

Architectural improvements:

- **Rotary position embeddings (RoPE)**: Better length generalization than learned positions
- **Flash Attention**: Memory-efficient attention (2--4x faster)
- **Grouped-query attention (GQA)**: Fewer KV heads for efficient inference

Training improvements:

- **Learning rate warmup + cosine decay**: Standard schedule for stable training
- **Mixed precision**: `torch.cuda.amp` for 2--3x speedup
- **Gradient accumulation**: Simulate larger batch sizes on limited hardware

Inference improvements:

- **KV cache**: Cache key/value tensors to avoid recomputation during generation
- **Quantization (INT8)**: 2--4x smaller model with minimal quality loss
- **Speculative decoding**: Use a small draft model to speed up a large model

Key takeaways

Core concepts from this lecture

1. **GPT is conceptually simple:** A stack of transformer decoder blocks predicting the next token
2. **Five core components:** Tokenizer → embeddings → masked attention → feed-forward → LM head
3. **Training requires care:** AdamW, gradient clipping, learning rate scheduling, and good data
4. **Generation is an art:** Temperature, top-k, and nucleus sampling control the creativity-coherence tradeoff
5. **Building it yourself** is the best way to deeply understand how modern LLMs work

Further reading

References and resources

- **Karpathy, "Let's build GPT"** -- Step-by-step video tutorial [[YouTube](#)]
- **nanoGPT** -- Clean, minimal GPT implementation [[GitHub](#)]
- **Radford et al. (2018)** -- "Improving Language Understanding by Generative Pre-Training" [[PDF](#)]
- **Vaswani et al. (2017)** -- "Attention is All You Need" [[arXiv](#)]
- **The Annotated Transformer** -- Harvard NLP [[Blog](#)]

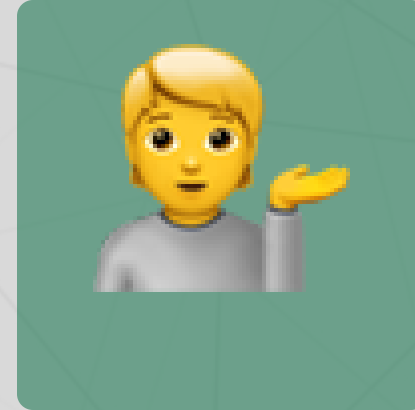
Questions?



Email



Discord



Office hours

Up next...

Week 9 (after break): Agents and tool use -- giving language models the ability to act